

Hospital Management System Backend

Developer Plan

Prepared on: 2026-06-02

1. Project Overview

The HMS backend is a Node.js Koa API using MongoDB through Mongoose. The current codebase follows a modular MVC-inspired structure where each business area has its own model, service, controller, and route files.

The API is versioned under `/api/v1` and already includes authentication, tenant management, doctors, staff, patients, beds, emergencies, appointments, admissions, discharges, and invoices.

2. Current Technology Stack

- Runtime: Node.js
- Framework: Koa
- Database: MongoDB
- ODM: Mongoose
- Authentication: JWT access token and refresh token
- Password hashing: bcryptjs
- API style: REST
- Deployment readiness: Docker and docker-compose files are present

3. Existing Folder Structure

- `config/`: environment and MongoDB connection configuration.
- `src/middlewares/`: authentication, authorization, and error handling middleware.
- `src/modules/`: feature modules.
- `src/utils/`: response and logging helpers.
- `docs/`: architecture and API documentation.

Each module generally follows this pattern: model, service, controller, and routes.

4. User Roles

Role	Current Status	Purpose
admin	Active	Platform owner. Can manage tenants and cross-tenant setup.
tenant	Active	Hospital or clinic owner/admin. Scoped to one tenant.
doctor	Active	Clinical user. Can view and manage patient workflow routes currently allowed to doctors.
staff	Active	Operational user such as receptionist, nurse, billing, or ward staff.
patient	Defined but not fully exposed	Patient login role exists in schema, but patient-facing routes are not yet implemented.

Recommendation: Use `patient` as the fourth primary user type if the product needs patient portal access. If the fourth type should be receptionist, nurse, accountant, or pharmacist, keep `staff` as the main role and add a `staff.role` or permission profile.

5. Current Key Data Models and Fields

User

Current fields: `name`, `email`, `password`, `role`, `tenantId`, `refreshToken`, `createdAt`.

Recommended additions: `status`, `phone`, `lastLoginAt`, `createdBy`, `updatedAt`, `passwordChangedAt`.

Tenant

Current fields: `name`, `state`, `country`, `metadata`, `createdAt`.

Recommended additions: `code`, `address`, `city`, `pincode`, `phone`, `email`, `registrationNumber`, `logoUrl`, `status`, `subscriptionPlan`, `settings`, `updatedAt`.

Doctor

Current fields: `name`, `specialization`, `phone`, `email`, `userId`, `tenantId`, `createdAt`.

Recommended additions: `doctorCode`, `department`, `qualification`, `registrationNumber`, `consultationFee`, `availability`, `status`, `updatedAt`.

Staff

Current fields: name, role, phone, email, tenantId, createdAt.

Recommended additions: staffCode, userId, department, designation, shift, permissions, status, updatedAt.

Patient

Current fields: userId, patientCode, name, dateOfBirth, gender, phone, email, address, emergencyContact, medicalHistory, tenantId, createdAt.

Recommended additions: bloodGroup, age, nationality, idProofType, idProofNumber, allergies, currentMedications, insuranceProvider, insurancePolicyNumber, status, updatedAt.

Bed

Current fields: bedNumber, ward, type, status, tenantId, assignedAdmissionId, createdAt.

Recommended additions: floor, roomNumber, dailyCharge, notes, compound unique index on tenantId + bedNumber, updatedAt.

Important: bedNumber is currently globally unique. In a multi-tenant system, this should usually be unique per tenant instead.

Appointment

Current fields: patientId, patientName, patientType, appointmentType, visitReason, doctorId, status, tenantId, createdBy, createdAt.

Recommended additions: appointmentCode, scheduledAt, department, tokenNumber, consultationFee, paymentStatus, notes, updatedAt.

Admission

Current fields: patientName, admissionType, appointmentId, bedNumber, doctorId, tenantId, status, admittedAt, dischargedAt.

Recommended additions: patientId, admissionCode, bedId, admissionReason, attendantName, attendantPhone, estimatedStayDays, notes, createdBy, updatedAt.

Emergency, Discharge, Invoice

Emergency should add case codes, triage notes, vitals, attendant details, and converted admission links. Discharge should add diagnosis, medications, follow-up, prepared by, approved by, and attachments. Invoice should add invoice number, patientId, line items, tax, discount, totals, paid amount, balance, payment reference, paid date, and createdBy.

6. Current API Modules

- Auth: login, refresh, register, profile.
- Tenant: create tenant, list tenants, show tenant.
- Doctor: create doctor, list doctors.
- Staff: create staff, list staff.
- Patient: create patient, list patients, show patient.
- Bed: create bed, list beds, list available beds.
- Emergency: create emergency case, list emergencies, admit emergency.
- Appointment: create appointment, list appointments.
- Admission: convert OPD to admission, create IPD admission, list admissions, discharge admission.
- Discharge: create discharge summary.
- Invoice: create invoice, list invoices, update payment status.

7. Multi-Tenant Strategy

Most business models include `tenantId`. Tenant users are expected to access only their own tenant data. Admin users can manage tenants.

Recommended improvements: add a shared tenant-scope helper, add compound indexes, make admin tenant selection consistent, force doctor and staff users to use token tenant scope, and never trust request body `tenantId` for non-admin users.

8. Role-Based Access Plan

Module	Admin	Tenant	Doctor	Staff	Patient
Tenants	Full	Own tenant only	No	No	No
Users	Full	Tenant	Limited profile	Limited profile	Own profile

Module	Admin	Tenant	Doctor	Staff	Patient
		users			
Doctors	Full	Full within tenant	Own profile	View	View assigned
Staff	Full	Full within tenant	View limited	Own profile	No
Patients	Full	Full within tenant	Assigned or tenant patients	Create/update operational fields	Own record
Appointments	Full	Full within tenant	Assigned schedule	Create/manage	Own appointments
Admissions	Full	Full within tenant	Assigned patients	Create/update workflow	Own admissions read-only
Beds	Full	Full within tenant	View	Manage	No
Emergency	Full	Full within tenant	Manage clinical status	Create/manage intake	No
Invoices	Full	Full within tenant	View limited	Create/update payment	Own invoices
Discharge	Full	Full within tenant	Create/approve	Assist	Own summary

9. Important Issues Found

1. `bed.service.js` defines `assignBedByNumber` and `releaseBedByNumber`, but does not export them. `admission.service.js` imports both functions, so admission bed assignment can fail at runtime.
2. `patient` role exists in the `User` model but is not currently allowed in patient-facing routes.
3. `staff.model.js` does not link to `User`, while doctor and patient records can link to users.
4. `bedNumber` and `patientCode` are globally unique. For multi-tenant HMS, these should usually be unique per tenant.
5. Admin tenant handling is inconsistent. Doctor routes support admin-selected `tenantId`, but many other controllers use `ctx.state.user.tenantId`, which may be empty for admin.
6. Input validation is minimal. Request bodies should be validated before writing to MongoDB.

7. There are no real automated tests yet. The current `npm test` is a placeholder.
8. Refresh tokens are stored as plain JWT strings. Consider hashing refresh tokens in production.

10. Recommended Backend Build Phases

Phase 1: Stabilize Foundation

- Fix bed service exports.
- Add request validation.
- Add common tenant-scope helper.
- Add timestamps to schemas.
- Add indexes for tenant-scoped uniqueness.
- Add basic integration tests for auth, tenant scoping, patient, bed, and admission workflows.

Phase 2: User and Permission System

- Finalize four or five user personas.
- Add user status and profile fields.
- Link staff records to user accounts.
- Add patient portal routes if patient login is required.
- Add permission profile support for staff sub-roles.

Phase 3: Core Hospital Workflow

- Improve patient registration, doctor schedules, appointment slots, OPD visits, prescriptions, vitals, diagnosis, admission lifecycle, and discharge approval.

Phase 4: Billing and Payments

- Add invoice line items, receipts, partial payments, refunds, reconciliation, and payment gateway or terminal integration.

Phase 5: Reporting and Audit

- Add audit logs, dashboards, exports, and operational reports for OPD, admissions, bed occupancy, and revenue.

Phase 6: Production Readiness

- Add environment-specific config, rate limiting, security headers, monitoring, backups, API

versioning policy, CI tests, and deployment pipeline.

11. Suggested Immediate Developer Tasks

1. Fix `bed.service.js` exports for `assignBedByNumber` and `releaseBedByNumber`.
2. Decide whether the fourth user type is `patient` or a staff sub-role such as receptionist.
3. Add validation middleware using a library such as Joi or Zod.
4. Add tenant-aware indexes to all tenant-scoped models.
5. Update admin behavior so admin can operate on selected tenant data consistently.
6. Add patient self-service routes if `patient` is part of the application scope.
7. Add API tests for login, tenant isolation, patient creation, appointment creation, bed assignment, admission, discharge, and invoice payment.

12. Recommended Fourth User Type

Based on the current codebase, the cleanest four-user setup is: `admin`, `tenant`, `doctor`, and `staff`.

Then define staff sub-roles such as receptionist, nurse, billing, pharmacist, lab technician, and ward manager. If a patient portal is required, keep `patient` as a fifth login role instead of forcing it into staff workflows.

13. Handoff Notes for Developers

- Keep modules small and business-focused.
- Do not remove `tenantId` from business documents.
- Use `ObjectId` references for cross-module relations where possible.
- Avoid storing only names when an entity reference exists. For example, admissions should store `patientId` and may also snapshot `patientName`.
- Add validation before adding more routes.
- Add tests before expanding complex workflows.
- Keep patient health data private and avoid exposing full data to roles that do not need it.